01. THÔNG TIN NHÓM VÀ MÔ HÌNH

BÁO CÁO GIỮA KỲ – NHÓM 15 (5 SINH VIÊN): ĂN HẾT THỨC ĂN RỒI TỚI G

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao

Giảng viên: TS. Nguyễn Bích Vân – IAI–UET–VNU

Lớp / Ngày giao / Ngày nộp: INT3401 / Ngày giao: 21/10/2026 / Ngày nộp: 29/10/2026

Môi trường Python 3.12 (Windows 11 / WSL Ubuntu)

Lệnh chạy: python run.py --batch --tag bat_buoc và python run.py --data du_lieu_nhom.json --batch --tag tu_tao

Danh sách thành viên

Thành viên / MSSV	Chủ trì	Hai test phụ trách	Kiểm tra chéo
1. Phan Ngọc Huy **MSSV:** 25022875	Phần A Bài B1 (M0) và thuật toán UCS	**1.** test_01_an_va_quay_lai **2.** test_02_mask_khong_them_bit	Kiểm dữ liệu riêng của TV4
2. Lê Đức Huy **MSSV:** 25022870	Bài B2 Thuật toán A*, hai heuristic h_F, h_V và vết A* trên H	**3.** test_03_qua_goal_som **4.** test_04_het_food_chua_toi_goal	Đối chiếu vết UCS của TV1
3. Hoàng Trung Kiên **MSSV:** 25022855	Chứng minh heuristic, bài B3–B4 và các phản ví dụ	**5.** test_05_khong_co_food_ban_dau **6.** test_06_no_path_va_limit	Đối chiếu heuristic của TV2
4. Nguyễn Quốc Hiếu **MSSV:** 25022853	Sinh dữ liệu riêng N1/N2 (seed 1526), chạy 24+12 cấu hình	**7.** test_07_dau_hoan_tat_ngan_sach_0 **8.** test_08_ban_do_sai_va_bien	Kiểm tình huống phản ví dụ của TV3
5. Dương Nhật Huy **MSSV:** 25022865	Tích hợp 10 kiểm thử, tái tạo kết quả, tổng hợp số liệu	**9.** test_09_khong_sua_input_va_trace **10.** test_10_kiem_duong_va_chi_phi	Kiểm trace và kết quả của TV4

1. Mô hình bài toán

• Ý nghĩa ký hiệu bản đồ: 'P' là vị trí xuất phát của Pacman, 'F' là viên thức ăn cần thu thập (tối đa 8 viên, m ≤ 8), 'G' là ô đích đến, '%' là tường chắn và '.' là ô trống đi được.

• Trạng thái & Hành động: Mỗi trạng thái là bộ s = (p, R), trong mã nguồn lưu thành bộ ba s = (r, c, mask) bao gồm vị trí hiện tại p = (r, c) ∈ V và tập thức ăn chưa ăn R ⊆ F_0 (mã hóa bằng số nguyên mặt nạ bit mask). Có 4 hành động di chuyển A = {R (Phải), D (Xuống), L (Trái), U (Lên)} với chi phí mỗi bước c(s, s') = 1.

• Chuyển trạng thái: Khi bước từ vị trí p sang ô hợp lệ kế tiếp q = (r', c'), trạng thái chuyển từ (p, R) sang (q, R \ {q}). Nghĩa là nếu ô q có thức ăn thì viên thức ăn đó tự động được ăn và bị loại khỏi tập còn lại R (xóa bit tương ứng về 0).

• Điều kiện hoàn tất (is_goal): Đạt đích khi Pacman đang đứng tại G và đã ăn hết toàn bộ thức ăn: p = G và R = ∅ (tương đương (r, c) = G và mask = 0).

• Vì sao vị trí đơn lẻ chưa đủ: Cùng một tọa độ nhưng tập thức ăn đã ăn khác nhau là hai trạng thái hoàn toàn khác nhau. Ví dụ trên bản đồ ['FPG']: Pacman xuất phát ở P=(0,1) khi còn F (mask = 1) → sang trái ăn F ở (0,0) (mask = 0) → bắt buộc phải quay lại ô P=(0,1) nhưng lúc này đã hết F (mask = 0) để đi tiếp sang G=(0,2). Nếu chỉ đánh dấu đã thăm theo tọa độ (r, c), ô (0,1) sẽ bị chặn ở lần quay lại.



Minh họa bản đồ [FPG]

• **Cận số trạng thái & Ý nghĩa:** Với V ô đi được và m viên thức ăn, có tối đa 2^m tập con thức ăn nên số trạng thái không vượt quá $V \times 2^m$. Tuy nhiên, cận đại số này chưa phản ánh tính liên thông thực tế (chưa loại trừ các ô hoặc thức ăn bị tường cô lập như ở M3, N2, hoặc các tổ hợp thức ăn không thể xảy ra do thứ tự đi qua hành lang).

2. Mặt nạ và phần việc tự làm

• **Thứ tự đánh số F và cách xóa bit:** Các viên thức ăn F được đánh số từ 0 đến m − 1 theo thứ tự hàng từ trên xuống dưới, cột từ trái sang phải. Trong số nguyên mask, bit thứ i bằng 1 nghĩa là viên F_i CHƯA ăn; bằng 0 là ĐÃ ăn. Khi bước vào ô chứa F_i, chương trình hạ bit thứ i từ 1 về 0 bằng phép toán bit AND với phần bù

• **Các tệp dùng nguyên:** Nhóm giữ nguyên các tệp mã nguồn chuẩn gồm maze.py, search.py, run.py, data_cases.py, tao_du_lieu.py, ucs_astar_toi_gian.py, chay_kiem_thu.py và dong_goi.py.

• **Phần việc nhóm tự thực hiện:** Sinh bộ dữ liệu riêng du_lieu_nhom.json (seed = 1526), viết mới tệp kiểm thử test_nhom.py (10 bài test), hoàn thiện phần TinhTay.pdf và chạy thực nghiệm phân tích 24 + 12 cấu hình.

02. THUẬT TOÁN VÀ HEURISTIC

3. Trình bày lại hai thuật toán

Công việc	UCS: mã giả / giải thích	A*: mã giả / giải thích
Khởi tạo biên / khóa	**Đặt hàm đánh giá:** h(s) = 0 với mọi trạng thái. **Khởi tạo OPEN:** Đưa trạng thái đầu vào hàng đợi ưu tiên heap với khóa (g, ticket, g, start), gán best_g[start] = 0, parent[start] = None.	**Đặt hàm đánh giá:** h(s) = h_F(s) hoặc h_V(s). **Khởi tạo OPEN:** Đưa trạng thái đầu vào heap với khóa (f, ticket, g, start) trong đó f = g + h(s), gán best_g[start] = 0.
Lấy phần tử kế tiếp	**Thao tác:** Lấy phần tử ở đỉnh min-heap (heappop). **Quy tắc:** Chọn trạng thái có chi phí g nhỏ nhất; nếu bằng nhau thì lấy phần tử vào hàng đợi sớm hơn (ticket nhỏ hơn – FIFO).	**Thao tác:** Lấy phần tử ở đỉnh min-heap (heappop). **Quy tắc:** Chọn trạng thái có tổng ước lượng f = g + h nhỏ nhất; nếu đồng hạng f thì ưu tiên ticket nhỏ hơn (FIFO).

Loại bản ghi cũ	**Điều kiện:** Kiểm tra nếu g ≠ best_g[s] thì tăng biến đếm stale và bỏ qua (continue). **Ý nghĩa:** Loại bỏ các bản ghi cũ có chi phí lớn hơn đường đi tốt nhất đã tìm thấy tới s.	**Điều kiện:** Kiểm tra nếu g ≠ best_g[s] thì tăng biến đếm stale và bỏ qua (continue). **Ý nghĩa:** Cơ chế xóa lười (lazy deletion) hoàn toàn giống UCS, chỉ xét bản ghi có g >= best_g[s].
Kiểm đích và ngân sách	**Thứ tự:** Tăng popped → kiểm tra đích is_goal(s) → kiểm tra ngân sách expanded ≥ max_expanded. **Ý nghĩa:** Kiểm đích ngay khi lấy ra khỏi heap giúp đảm bảo đường đi tối ưu.	**Thứ tự:** Tăng popped → kiểm tra đích is_goal(s) → kiểm tra ngân sách expanded ≥ max_expanded. **Ý nghĩa:** Kiểm đích khi pop (không dừng sớm lúc mới sinh con); nếu chạm ngưỡng ngân sách thì trả về 'limit'.
Sinh con / cập nhật mask	**Thao tác:** Tăng expanded; duyệt 4 hướng theo thứ tự RDLU hoặc DRUL. **Cập nhật mask:** Nếu ô kế tiếp q hợp lệ và chứa F_i thì hạ bit thứ i trong mask về 0, chi phí bước mới ng = g + 1.	**Thao tác:** Tăng expanded; duyệt 4 hướng theo thứ tự RDLU hoặc DRUL. **Cập nhật mask:** Sinh trạng thái kế tiếp v = (r', c', mask mới) và tăng ng = g + 1 giống hệt UCS.
Cập nhật g và cha	**Điều kiện:** Chỉ cập nhật khi tìm được đường ngắn hơn hẳn: ng < best_g[v]. **Thao tác:** Gán best_g[v] = ng, parent[v] = (s, action) và đẩy bản ghi mới với khóa f = ng vào heap.	**Điều kiện:** Chỉ cập nhật khi tìm được đường ngắn hơn hẳn: ng < best_g[v]. **Thao tác:** Gán best_g[v] = ng, parent[v] = (s, action) và đẩy bản ghi mới với khóa f = ng + h(v) vào heap.
Dựng đường / kết thúc	**Khi gặp đích:** Lần ngược bảng parent[s] từ trạng thái đích về trạng thái đầu rồi đảo ngược danh sách. **Khi biên rỗng:** Nếu heap rỗng mà chưa gặp đích, kết luận 'no_path' và cost = None.	**Khi gặp đích:** Lần ngược bảng parent[s] từ trạng thái đích về trạng thái đầu để lấy chuỗi states và actions. **Khi biên rỗng:** Nếu heap rỗng mà chưa gặp đích, kết luận 'no_path' và cost = None.

• **Tên tệp và hàm tương ứng:** Thuật toán tìm kiếm được cài đặt trong hàm solve() thuộc tệp search.py, gọi các hàm mô hình mê cung initial(), successors(), is_goal() và heuristic() trong tệp maze.py.

.

• **Quy tắc đồng hạng & Khi nào đổi cha:** Mỗi phần tử trong heap là bộ (f, ticket, g, s). Khi hai trạng thái có cùng giá trị f, thuật toán lấy phần tử có ticket nhỏ hơn (tức là được chèn vào biên OPEN sớm hơn theo thứ tự hướng RDLU/DRUL). Chương trình chỉ đổi cha parent[v] và cập nhật best_g[v] khi chi phí mới nhỏ hơn hẳn chi phí cũ (ng < best_g[v]), không đổi cha khi bằng giá.

4. Hai heuristic

• **Định nghĩa khoảng cách Manhattan:** Với hai ô x = (r_x, c_x) và y = (r_Y, c_Y), khoảng cách Manhattan là d(x, y) = $|r_x - r_Y| + |c_x - c_Y|$.

• **Hàm heuristic food (h_F):** h_F(p, R) = max_{f ∈ R} d(p, f) nếu tập thức ăn còn lại R ≠ ∅, và h_F(p, ∅) = 0. Bằng lời: h_F là khoảng cách Manhattan từ vị trí hiện tại p tới viên thức ăn còn lại nằm xa nhất.

• **Hàm heuristic via (h_V):** h_V(p, R) = max({d(p, G)} ∪ {d(p, f) + d(f, G) : f ∈ R}). Bằng lời: h_V là giá trị lớn nhất giữa khoảng cách trực tiếp từ p tới đích G và tổng khoảng cách từ p đi qua một viên thức ăn f còn lại rồi tới G.

• **Khi hết thức ăn nhưng chưa tới đích (R = ∅, p ≠ G):** Hàm h_F trả về 0 (mất định hướng về G, làm A* suy biến thành UCS ở chặng cuối), trong khi hàm h_V trả về d(p, G) > 0, tiếp tục dẫn đường thẳng về đích G.

• **Chứng minh chấp nhận được (Admissible):** Vì mỗi bước chỉ đi ngang hoặc dọc 1 ô và việc bỏ qua tường chắn chỉ làm đường đi ngắn hơn hoặc bằng thực tế, nên d(x, y) luôn là cận dưới của chi phí thực tế giữa hai ô. Để hoàn thành bài toán từ trạng thái s = (p, R), mọi đường đi hợp lệ đều phải đi từ p tới từng viên f ∈ R rồi từ f đi tiếp tới đích G, do đó chi phí tối ưu thực tế h*(s) luôn thỏa mãn: h*(s) ≥ d(p, f) + d(f, G) ≥ d(p, f) với mọi f ∈ R, đồng thời h*(s) ≥ d(p, G). Lấy giá trị lớn nhất trên tập hữu hạn các cận dưới này, ta luôn có 0 ≤ h_F(s) ≤ h_V(s) ≤ h*(s) (cả hai đều chấp nhận được).

• **Nhất quán khi bước đi không ăn F (R' = R):** Xét bước chuyển từ p sang ô kề p' với chi phí c = 1 (nên d(p, p') = 1). Theo bất đẳng thức tam giác của khoảng cách Manhattan, với mọi viên f ∈ R ta có: d(p, f) ≤ d(p, p') + d(p', f) = 1 + d(p', f), và d(p, G) ≤ 1 + d(p', G). Cộng d(f, G) vào hai vế và lấy max trên R, ta được h_F(s) ≤ 1 + h_F(s') và h_V(s) ≤ 1 + h_V(s').

• **Nhất quán khi bước đi ăn viên F (kể cả viên cuối):** Giả sử bước sang ô kề p' ăn viên thức ăn f_0 (tức là p' = f_0 và R' = R \ {f_0}). Với các viên f còn lại trong R', bất đẳng thức tam giác vẫn giữ nguyên. Xét riêng tại viên vừa ăn f_0 = p': vì p kề p' nên d(p, f_0) = 1. Do đó: với h_F, ta có d(p, f_0) = 1 ≤ 1 + h_F(s') (kể cả khi f_0 là viên thức ăn cuối cùng làm R' = ∅ và h_F(s') = 0 thì 1 ≤ 1 + 0 vẫn đúng). Với h_V, ta có d(p, f_0) + d(f_0, G) = 1 + d(p', G) ≤ 1 + h_V(s') (vì theo định nghĩa h_V(s') luôn ≥ d(p', G)). Vậy cả h_F và h_V đều nhất quán.

• **Quan hệ h_V ≥ h_F và giới hạn kết luận về thời gian:** Vì d(f, G) ≥ 0 và d(p, G) ≥ 0 nên luôn có h_V(s) ≥ h_F(s) ≥ 0 tại mọi trạng thái, giúp A* dùng h_V mở rộng ít trạng thái hơn hoặc bằng h_F và UCS. Tuy nhiên không thể kết luận h_V luôn chạy nhanh hơn về thời gian thực tế (mili-giây) trên các bản đồ nhỏ, vì mỗi lần tính h_V phải duyệt qua các viên F còn lại tốn O(m) phép tính và thời gian thực thi dưới 1 ms chịu nhiễu lớn từ hệ điều hành.

5. Độ phức tạp

• **Phân biệt ô vật lý và trạng thái (V, V', E'):** Bản đồ có V ô trống đi được, nhưng đồ thị không gian trạng thái G' = (V', E') có số đỉnh trạng thái s = (r, c, mask) là $|V'| \le V \times 2^m$ và số cung chuyển trạng thái (tối đa 4 hướng mỗi đỉnh) là $|E'| \le 4 \times V \times 2^m$.

• **Độ phức tạp Thời gian, Bộ nhớ & Chi phí trace:** Nhờ heuristic nhất quán, mỗi trạng thái chỉ mở rộng tối đa 1 lần (reopened = 0). Độ phức tạp thời gian trường hợp xấu nhất là $O(|E'| \times (m + \log |V'|)) = O(V \times 2^m \times (m + \log V))$ và bộ nhớ là $O(|V'|) = O(V \times 2^m)$. Khi bật ghi vết (trace) K bước, hàm snapshot() sao chép và sắp xếp lại toàn bộ heap ở mỗi bước nên tốn thêm thời gian và bộ nhớ O(K × |heap| × log |heap|).

• **Các giả thiết bảo đảm tối ưu:**

- (1) Đồ thị trạng thái hữu hạn.
- (2) Chi phí mỗi bước đi dương và đồng nhất c = 1.
- (3) Kiểm tra điều kiện đích ngay khi lấy trạng thái ra khỏi hàng đợi ưu tiên (pop).
- (4) Hàm heuristic chấp nhận được ($0 \le h(s) \le h^*(s)$).

03. TÍNH TAY VÀ ĐỐI CHIẾU MÃ

• **Nội dung tệp TinhTay.pdf (trang B1–B4):** Trang B1 (Phân tích M0 và chứng minh tối ưu), Trang B2 (Bảng 6 lần lấy đầu của UCS trên H), Trang B3 (Bảng 6 lần lấy đầu của A_V trên H), Trang B4 (Chứng minh heuristic, phản ví dụ B3–B4 và tính tay 3 test).

• **Thời điểm ghi dự đoán:** Nhóm hoàn thành lập bảng tính tay và ghi dự đoán vào ngày 25/10/2026 trước khi chạy lệnh đối chiếu với chương trình.

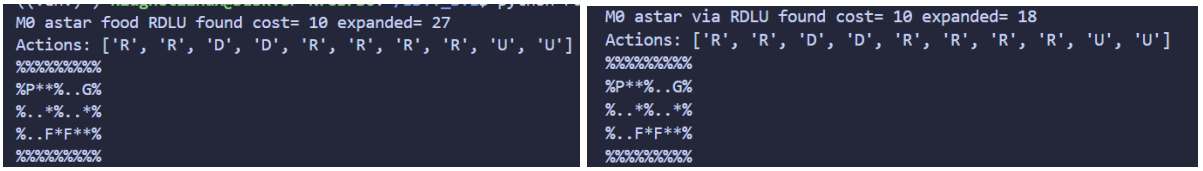
6. M0

• **Thông số bản đồ M0:** Kích thước 5×9; xuất phát P = (1,1), đích G = (1,7), có m = 2 viên thức ăn là a = (3,3) và b = (3,5). Số ô đi được là V = 19 ô; cận trên số trạng thái là $V \times 2^m = 19 \times 2^2 = 76$ trạng thái.

• **Giá trị h_F, h_V ở đầu và hai con:** Tại trạng thái đầu s_0 = (1,1; ab) (mask = 3): d(P, a) = 4, d(P, b) = 6, d(P, G) = 6, d(a, G) = 6, d(b, G) = 4 ⇒ $h_F(s_0) = \max(4, 6) = 6$ và $h_V(s_0)$ =

max(6, 4+6, 6+4) = 10. Hai trạng thái con hợp lệ từ s_0 là: đi Phải tới (1,2; ab) có g = 1, h_F = 5, h_V = 9 (f_F = 6, f_V = 10); đi Xuống tới (2,1; ab) có g = 1, h_F = 5, h_V = 9 (f_F = 6, f_V = 10).

• **Đường đi tối ưu & Chứng minh bằng cận dưới:** Với thứ tự RDLU, lộ trình tìm được gồm 10 bước [R, R, D, D, R, R, R, R, U, U]: đi từ (1,1) xuống ăn a=(3,3) ở bước 4, sang ăn b=(3,5) ở bước 6, rồi đi lên đích G=(1,7) ở bước 10 (chi phí cost = 10). Vì h_V là hàm chấp nhận được và ngay tại đầu đã có $h_V(s_0) = 10$, mọi đường đi hợp lệ đều phải tốn ít nhất 10 bước; lộ trình tìm được đạt đúng cận dưới 10 nên chắc chắn tối ưu.



Kết quả chạy M0 với 2 heuristic food và via

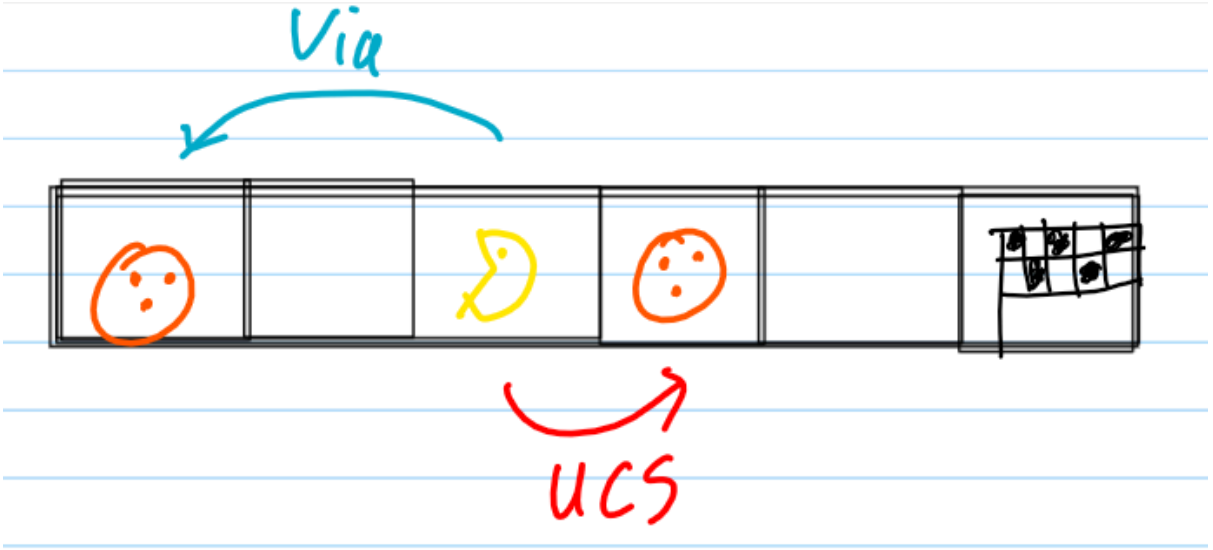
7. Sáu lần lấy đầu trên H

Lần	UCS: state / g	A_V: state / g,h,f	Khớp tay–mã?
1	**(1,3,3)** [tức (3; ab)] **g = 0**	**(1,3,3)** [tức (3; ab)] **g = 0, h = 7, f = 7**	Khớp 100%
2	**(1,4,1)** [tức (4; a)] **g = 1**	**(1,2,3)** [tức (2; ab)] **g = 1, h = 6, f = 7**	Khớp 100%
3	**(1,2,3)** [tức (2; ab)] **g = 1**	**(1,1,2)** [tức (1; b)] **g = 2, h = 5, f = 7**	Khớp 100%
4	**(1,5,1)** [tức (5; a)] **g = 2**	**(1,2,2)** [tức (2; b)] **g = 3, h = 4, f = 7**	Khớp 100%
5	**(1,3,1)** [tức (3; a)] **g = 2**	**(1,3,2)** [tức (3; b)] **g = 4, h = 3, f = 7**	Khớp 100%
6	**(1,1,2)** [tức (1; b)] **g = 2**	**(1,4,0)** [tức (4; -)] **g = 5, h = 2, f = 7**	Khớp 100%

• **Bảng OPEN đầy đủ, con mới và cha:** Trình bày chi tiết tại TinhTay.pdf trang B2 (cho UCS) và trang B3 (cho A_V), đối chiếu khớp hoàn toàn với output/tay_H_UCS.json và output/tay_H_Avia.json.

• **Lần đầu lấy khác & Nguyên nhân:** Hai thuật toán lấy khác nhau ngay ở bước thứ 2: UCS lấy (1,4,1) (đi phải ăn b trước), còn A_V lấy (1,2,3) (đi trái về phía a trước). Nguyên nhân: Ở

bước 1, hướng R được xét trước nên sinh ra (1,4,1) với ticket = 1, g = 1; hướng L sinh ra (1,2,3) với ticket = 2, g = 1. UCS chỉ xét g = 1 bằng nhau nên lấy ticket nhỏ hơn là (1,4,1). Ngược lại, A_V tính được tại (1,4,1) vẫn phải quay về a=(1,1) rồi mới tới G=(1,6) nên h_V = 3 + 5 = 8 ⇒ f = 9; còn tại (1,2,3) có h_V = 1 + 5 = 6 ⇒ f = 7 < 9 nên A_V chọn (1,2,3).

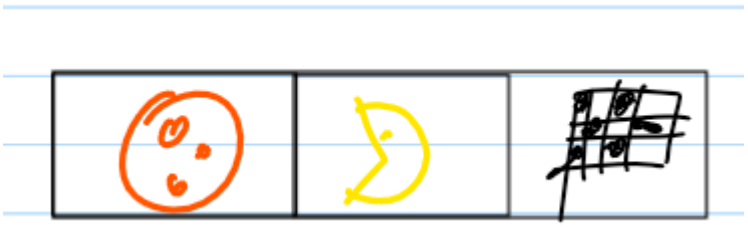


Minh họa lựa chọn của 2 thuật toán tại bước 2

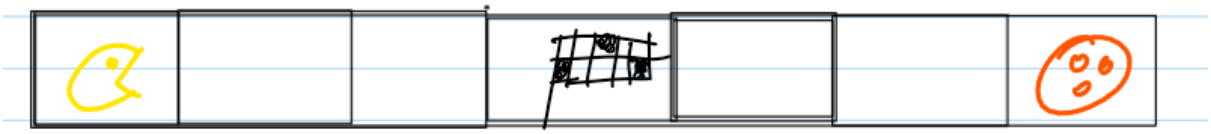
• **Đường hoàn chỉnh & Chi phí tối ưu:** Trên bản đồ H (P=(1,3), a=(1,1), b=(1,4), G=(1,6)), đường đi tối ưu là ['L', 'L', 'R', 'R', 'R', 'R', 'R'] (sang trái ăn a ở bước 2, quay lại qua ô (1,2) và (1,3) ở trạng thái mới chỉ còn b, ăn b ở bước 5 và tới đích G ở bước 7). Chi phí tối ưu cost = 7, đạt đúng cận dưới $h_V(s_0) = 7$.

8. Các phản ví dụ

• **Ô cần quay lại & Lỗi gộp vị trí visited = {(r, c)}:** Trên bản đồ H (hoặc ['FPG']), sau khi sang trái ăn viên a=(1,1), Pacman bắt buộc phải đi ngược lại qua ô (1,2) và (1,3) để sang phải về G. Nếu chỉ lưu visited theo tọa độ (r, c), hai ô này đã bị đánh dấu thăm lúc đi sang trái nên đường quay về bị chặn, làm thuật toán báo sai là không có lời giải (được kiểm tra bởi test_01_an_va_quay_lai).



• **Qua G sớm & Hết F nhưng chưa tới G:** Trên hành lang M2 (%P..G..F%), Pacman không được dừng ở lần đầu đi ngang qua G vì lúc đó vẫn còn thức ăn F (trên đường tối ưu cost = 9, Pacman ghé ô G tổng cộng 2 lần: lần 1 ở bước 3 với mask = 1, lần 2 ở bước 9 với mask = 0). Ngược lại, khi vừa ăn viên F cuối cùng nhưng còn cách G 3 bước thẳng: lúc này h_F = 0, h_V = 3 và điều kiện đích vẫn là False; nếu viết lệnh if h == 0: return success thì A* với h_F sẽ dừng sai ngay tại ô thức ăn cuối.



• **Hai heuristic sai trên hành lang PFFG:** Xét bản đồ ['PFFG'] (P=(0,0), F_1=(0,1), F_2=(0,2), G=(0,3)) có chi phí tối ưu thực tế là 3 bước. (1) Nếu dùng hàm tổng h_sum = d(P,G) + ∑ d(P, f) thì tại đầu h_sum = 3 + (1 + 2) = 6 > 3, vượt quá chi phí thực tế nên không phải là cận dưới chấp nhận được. (2) Nếu tính h_F luôn dựa trên tập thức ăn ban đầu F_0 thay vì tập còn lại R, thì khi đã ăn hết và đứng tại G vẫn có h_F = max(d(G, F_1), d(G, F_2)) = 2 ≠ 0, vi phạm điều kiện bắt buộc h(goal) = 0.



• **Phân biệt no_path, limit và Ngân sách 0 tại đích:** no_path là khi hàng đợi OPEN đã rỗng mà vẫn không tới được đích (bản đồ thực sự vô nghiệm); còn limit là khi dừng sớm do số lần mở rộng chạm ngưỡng max_expanded. Nếu đặt ngân sách max_expanded = 0 nhưng trạng thái bắt đầu đã thỏa mãn cả hai điều kiện đích (đứng tại G và mask = 0), thuật toán kiểm đích trước khi kiểm ngân sách nên vẫn trả về status = 'found' và cost = 0.

• **Điều hiểu rõ hơn sau đối chiếu:** Trước khi đối chiếu trace trên bản đồ H, nhóm từng nghĩ khi đồng hạng f thì heap sẽ so sánh tọa độ tuple trạng thái; sau khi đọc mã nguồn search.py, nhóm hiểu rõ heap so sánh thành phần thứ hai là ticket (thứ tự sinh nút theo RDLU/DRUL), do đó nhánh R được sinh trước sẽ được UCS lấy ra trước nhánh L.

04. THỰC NGHIỆM CHUNG VÀ RIÊNG

• **Lệnh chạy thực nghiệm & Ngân sách:** Chạy python run.py --batch --tag bat_buoc (24 cấu hình chung) và python run.py --data du_lieu_nhom.json --batch --tag tu_tao (12 cấu hình riêng) với ngân sách mặc định max_expanded = 200000.

• **Tệp dữ liệu đầu ra:** Lưu tại output/bat_buoc.json, output/bat_buoc.csv và output/tu_tao.json, output/tu_tao.csv. (Ký hiệu N là số trạng thái mở rộng expanded; các ca vô nghiệm no_path giữ nguyên cost = None).

9. Tóm tắt 24 cấu hình chung

Bản đồ / thứ tự	Status / cost	N UCS	N A_F	N A_V
M0 / RDLU	found (cost = 10)	30 (peak = 6)	**27** (peak = 8) **S_F = 10.0%**	**18** (peak = 6) **S_V = 40.0%**
M0 / DRUL	found (cost = 10)	30 (peak = 6)	**27** (peak = 7) **S_F = 10.0%**	**18** (peak = 6) **S_V = 40.0%**
M1 / RDLU	found (cost = 13)	43 (peak = 5)	**42** (peak = 8) **S_F = 2.3%**	**30** (peak = 8) **S_V = 30.2%**
M1 / DRUL	found (cost = 13)	43 (peak = 5)	**42** (peak = 8) **S_F = 2.3%**	**30** (peak = 8) **S_V = 30.2%**
M2 / RDLU	found (cost = 9)	9 (peak = 1)	**9** (peak = 1) **S_F = 0.0%**	**9** (peak = 1) **S_V = 0.0%**
M2 / DRUL	found (cost = 9)	9 (peak = 1)	**9** (peak = 1) **S_F = 0.0%**	**9** (peak = 1) **S_V = 0.0%**
M3 / RDLU	no_path (cost = None)	11 (peak = 4)	**11** (peak = 4) **S_F = 0.0%**	**11** (peak = 5) **S_V = 0.0%**
M3 / DRUL	no_path (cost = None)	11 (peak = 5)	**11** (peak = 5) **S_F = 0.0%**	**11** (peak = 5) **S_V = 0.0%**

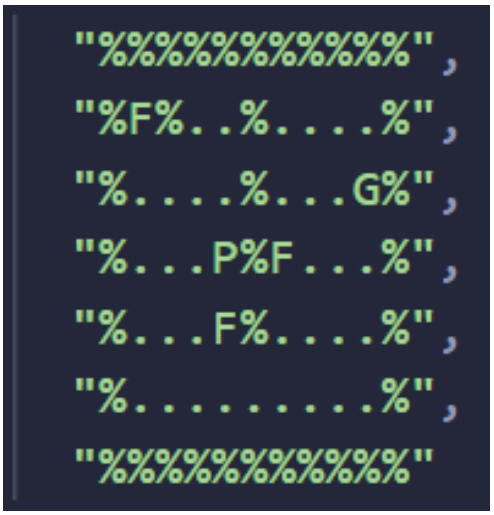
• **Công thức tỷ lệ giảm mở rộng & Xử lý mẫu số 0:** Với N_UCS > 0, tỷ lệ giảm số nút mở rộng được tính bằng S_F = (1 − N_AF / N_UCS) × 100% và S_V = (1 − N_AV / N_UCS) × 100%. Nếu mẫu số N_UCS = 0 (khi trạng thái đầu đã là đích), ta ghi là 'không áp dụng' để tránh phép chia cho 0.

• **Đỉnh biên & Cùng chi phí nhưng khác đường đi:** Theo tệp output/bat_buoc.csv, số bản ghi lớn nhất trong heap (frontier_peak) đạt tối đa là 8 (ở M0 và M1). Có hai trường hợp điển hình tìm được cùng chi phí tối ưu nhưng lộ trình khác nhau: (1) Trên M0 (cost = 10), thứ tự RDLU đi ngang sang phải trước [R,R,D,D,R,R,R,R,U,U] còn DRUL đi xuống trước [D,D,R,R,R,R,R,R,U,U]; (2) Trên M1 (cost = 13), UCS và A_F đi xuống ăn viên F ở (3,4) trước rồi mới lên ăn viên (1,1), trong khi A_V nhờ tính cả khoảng cách tới đích G nên chọn ăn viên bên trái (1,1) trước rồi mới xuống (3,4) và đi thẳng về G.

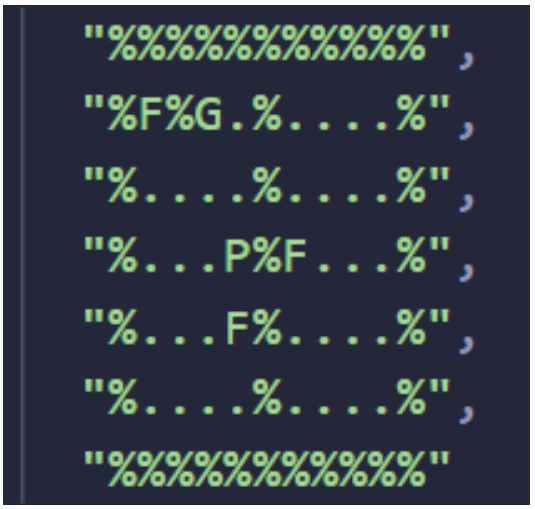
10. N1/N2: 12 cấu hình riêng

Bản đồ / thứ tự	Status / cost	N UCS	N A_F	N A_V
N1 / RDLU	found (cost = 20)	264 (peak = 25)	**203** (peak = 38) **S_F = 23.1%**	**80** (peak = 16) **S_V = 69.7%**

N1 / DRUL	found (cost = 20)	273 (peak = 25)	**205** (peak = 39) **S_F = 24.9%**	**80** (peak = 16) **S_V = 70.7%**
N2 / RDLU	no_path (cost = None)	72 (peak = 11)	**72** (peak = 12) **S_F = 0.0%**	**72** (peak = 12) **S_V = 0.0%**
N2 / DRUL	no_path (cost = None)	72 (peak = 10)	**72** (peak = 12) **S_F = 0.0%**	**72** (peak = 12) **S_V = 0.0%**



N1



N2

• **Thông số bộ dữ liệu riêng (seed = 1526):** Sinh bằng lệnh python tao_du_lieu.py --seed 1526, gồm hai bản đồ N1 và N2 kích thước 7 hàng × 11 cột. Cả hai đều có P = (3,4) và 3 viên thức ăn tại F_0 = (1,1) (trong ngõ cụt góc trên bên trái), F_1 = (3,6) (ở nửa phòng bên phải) và F_2 = (4,4) (ngay dưới P).

• **Bản đồ N1 (có lời giải, cost = 20):** Đích G = (2,9) nằm ở phòng bên phải và cửa thông giữa hai phòng tại ô (5,5) mở. Lộ trình tối ưu RDLU gồm 20 bước: từ P=(3,4) đi lên vào ngõ cụt (1,1) ăn F_0, lùi ra và quay lại qua các ô (2,1), (2,2), (2,3), (2,4), (3,4) để xuống (4,4) ăn F_2, sau đó đi qua cửa (5,5) sang phòng phải ăn F_1=(3,6) rồi tiến tới G=(2,9). Trên N1, A_V giảm tới ~70% số lần mở rộng so với UCS.

• **Bản đồ N2 (tới G được nhưng F bị ngăn cách):** Đích G = (1,3) được chuyển sang phòng bên trái (cùng phía với P, chỉ cách P đúng 3 bước đi), nhưng cửa tại ô (5,5) bị đóng thành tường '%'. Bức tường dọc cột 5 chia cắt hoàn toàn miền bên trái (19 ô trống chứa P, G, F_0, F_2) với miền bên phải (chứa viên F_1 tại (3,6)). Vì không thể sang miền phải ăn F_1, cả 3 thuật toán đều phải duyệt cạn đúng $(19 - 2)\times 2^2 + 2\times 2^1 = 72$ trạng thái miền trái rồi kết luận no_path.

• **Thí nghiệm thay F/G trên bản sao N2:** Trên bản sao của N2, nhóm thử dời viên F_1 từ (3,6) sang ô (2,3) ở miền bên trái (hoặc mở lại cửa (5,5)). Dự đoán: bài toán trở thành có lời giải với cost = 11 (từ P=(3,4) xuống ăn (4,4) mất 1 bước → lên ăn (2,3) mất 3 bước → vào ăn (1,1) mất 3 bước → vòng ra G=(1,3) mất 4 bước). Kết quả chạy thực tế khớp 100%: status = 'found', cost = 11, số mở rộng N(A_V) = 19 < N(A_F) = 36 < N(UCS) = 48.

05. KIỂM THỬ NHÓM TỰ VIẾT

• **Lệnh chạy & Kết quả log thực tế:** Chạy lệnh python chay_kiem_thu.py để thực thi tệp test_nhom.py và xuất log vào output/ket_qua_test.txt. Kết quả: 10/10 bài kiểm thử đều đạt (OK, 0 lỗi).

• **Ba bài kiểm thử có kỳ vọng tính tay:** Gồm Test 1 (test_01_an_va_quay_lai), Test 3 (test_03_qua_goal_som) và Test 5 (test_05_khong_co_food_ban_dau), được tính tay chi tiết tại Trang B4 của tệp TinhTay.pdf.

STT	Tên test / dữ liệu	Kỳ vọng và cách suy ra	Thực tế	Người
1	**test_01_an_va_quay_lai** **Bản đồ:** ['%%%%%%%%', '%F.P.G%', '%%%%%%%%']	Tính tay (TinhTay.pdf B4): Sang trái 2 bước ăn F=(1,1) rồi quay lại 4 bước qua P tới G=(1,5) ⇒ cost = 6, actions = ['L','L','R','R','R','R'].	**PASS (cost = 6)**	Phan Ngọc Huy
2	**test_02_mask_khong_them_bit** **Bản đồ:** ['PF.G'] (F tại (0,1))	Sau khi bước vào (0,1) ăn F làm mask về 0, lùi lại (0,0) rồi bước vào (0,1) lần hai thì mask vẫn bằng 0 (không bật lại thức ăn).	**PASS (mask = 0)**	Phan Ngọc Huy
3	**test_03_qua_goal_som** **Bản đồ:** ['%%%%%%%%', '%P.G..F%', '%%%%%%%%']	Tính tay (TinhTay.pdf B4): Đi qua G=(1,3) khi mask = 1 chưa được dừng; phải tới (1,6) ăn F rồi lùi 3 bước về G ⇒ cost = 8.	**PASS (cost = 8)**	Lê Đức Huy
4	**test_04_het_food_chua_toi_goal** **Bản đồ:** ['%%%%%%%%', '%P.F.G%', '%%%%%%%%']	Tại trạng thái s=(1,3,0) vừa ăn hết F nhưng chưa tới G=(1,5): is_goal(s) = False, h_F(s) = 0 nhưng h_V(s) = 2.	**PASS (h_F=0, h_V=2)**	Lê Đức Huy

5	**test_05_khong_co_food_ban_dau** **Bản đồ:** ['%%%%%%%%', '%P...G%', '%%%%%%%%']	Tính tay (TinhTay.pdf B4): Bản đồ không có F (mask đầu = 0), tại P có h_F = 0, h_V = 4; đi thẳng 4 bước R tới G ⇒ cost = 4.	**PASS (cost = 4)**	Hoàng Trung Kiên
6	**test_06_no_path_va_limit** **Bản đồ:** ['%P%G%F%'] & M0 (budget=5)	Khi F bị tường chắn trả về status = 'no_path', cost = None; khi chạy M0 với max_expanded = 5 thì dừng sớm với status = 'limit'.	**PASS (no_path/ limit)**	Hoàng Trung Kiên
7	**test_07_dau_hoan_tat_ngan_sach_0** **Bản đồ:** ['PFG'], start=(0,2,0), budget=0	Trạng thái đầu đã ở đích G và mask = 0: dù đặt max_expanded = 0 vẫn trả về status = 'found', cost = 0, popped = 1, expanded = 0.	**PASS (cost = 0)**	Nguyễn Quốc Hiếu
8	**test_08_ban_do_sai_va_bien** **Bản đồ:** Lỗi ký tự, lệch hàng, >8 F, ['PG']	Hàm parse_maze từ chối dữ liệu sai bằng ValueError; trên ['PG'] kiểm tra không đi chéo và không quấn mép biên.	**PASS (ValueError)**	Nguyễn Quốc Hiếu
9	**test_09_khong_sua_input_va_trace** **Bản đồ:** H với trace=0/4 và h='zero'	Mảng rows đầu vào không bị sửa sau khi chạy; bật/tắt trace không đổi kết quả; A* với h='zero' có pop_order giống hệt UCS.	**PASS (Bất biến)**	Dương Nhật Huy
10	**test_10_kiem_duong_va_chi_phi** **Bản đồ:** M0, M1, H và N1	Mô phỏng bước đi trên path: mọi bước kề nhau, không xuyên tường, ăn đủ F, kết thúc tại G và cả 3 cấu hình đồng ý cost tối ưu.	**PASS (Khớp 100%)**	Dương Nhật Huy

11. Phạm vi kiểm thử

• **Ăn F tự động, quay lại ô cũ & Mask chỉ xóa bit:** Được kiểm tra bởi test_01_an_va_quay_lai và test_02_mask_khong_them_bit.

• **Qua G khi còn F, ăn hết chưa tới G & Không có F ban đầu:** Được kiểm tra bởi test_03_qua_goal_som, test_04_het_food_chua_toi_goal và test_05_khong_co_food_ban_dau.

• **Vô nghiệm (no_path), Quá ngân sách (limit) & Đầu hoàn tất với ngân sách 0:** Được kiểm tra bởi test_06_no_path_va_limit và test_07_dau_hoan_tat_ngan_sach_0.

• **Bản đồ sai, chỉ số biên, tường chắn & Tính hợp lệ của đường đi:** Được kiểm tra bởi test_08_ban_do_sai_va_bien và test_10_kiem_duong_va_chi_phi.

• **Đổi F/G, không sửa đầu vào, bật/tắt trace & Tương đương h=0 với UCS:** Được kiểm tra bởi test_09_khong_sua_input_va_trace.

• **Kiểm chứng lộ trình và sự đồng thuận chi phí tối ưu:** Được kiểm tra bởi test_10_kiem_duong_va_chi_phi trên các bản đồ M0, M1, H và N1.

06. NHẬN XÉT VÀ KIỂM TRA TRƯỚC NỘP

13. Ba nhận xét có minh chứng

Nhận xét 1 – Mô hình trạng thái và ảnh hưởng của số thức ăn : Biểu diễn trạng thái bằng bộ ba (r, c, mask) là điều kiện bắt buộc để phân biệt các lần ghé thăm cùng một tọa độ với tập thức ăn còn lại khác nhau. Thực nghiệm trên M1 (cost = 13), M2 (cost = 9), H (cost = 7), N1 (cost = 20) và test_01_an_va_quay_lai chứng minh nếu chỉ lưu tọa độ (r, c), tác tử sẽ bị kẹt ngay khi vào ngõ cụt ăn thức ăn. Tuy nhiên, mỗi viên F tăng thêm làm không gian trạng thái nhân đôi theo hàm mũ $V \times 2^m$: từ M0 (V = 19, m = 2, cận 76, UCS mở rộng 30 nút) tăng vọt lên N1 (V = 44, m = 3, cận 352, UCS mở rộng 264–273 nút).

Nhận xét 2 – So sánh h_F, h_V và UCS : Việc đưa nghĩa vụ tới đích G vào hàm đánh giá giúp h_V luôn trội hơn h_F ($h_V \geq h_F \geq 0$), nhờ đó cắt giảm mạnh số nút mở rộng mà vẫn bảo toàn chi phí tối ưu trên mọi bản đồ. Điển hình trên N1/RDLU, A_V chỉ mở rộng 80 nút, giảm 60.6% so với A_F (203 nút) và giảm 69.7% so với UCS (264 nút), bởi h_F bằng 0 ngay khi ăn viên cuối trong khi h_V vẫn bằng $d(p, G) > 0$ (test_04). Khi đồng hạng f, thứ tự RDLU/DRUL quyết định ticket FIFO nên có thể chọn lộ trình tối ưu khác nhau (như trên M0). Dù tính h_V tốn O(m) mỗi nút, lợi ích giảm không gian duyệt vẫn vượt trội.

Nhận xét 3 – Điều kiện đích, thức ăn bị ngăn và giới hạn tài nguyên : Điều kiện hoàn tất yêu cầu thỏa mãn đồng thời vị trí (r, c) = G và mask = 0, giúp thuật toán không dừng sớm khi đi ngang qua G lúc còn thức ăn (minh chứng ở M2 và test_03) hoặc dừng tại viên F cuối (test_04). Đặc biệt, khi một viên F bị tường cô lập như ở M3 (mở rộng 11 nút) và N2 (tới G rất gần nhưng F_1 bị tường cột 5 ngăn cách, mở rộng đúng 72 trạng thái miền trái), cả UCS, A_F và A_V đều phải duyệt cạn toàn bộ thành phần liên thông trước khi trả về no_path (cost

= None), khác hẳn với trạng thái limit khi dừng sớm do hết ngân sách max_expanded (test_06).

14. Tái tạo và bảo vệ

• Hai giới hạn của thí nghiệm: (1) Số lượng thức ăn giới hạn ở m ≤ 8 và bản đồ nhỏ nên chưa bộc lộ hiện tượng bùng nổ tổ hợp khi m lớn; (2) Thời gian chạy thực tế dưới 1 ms chịu nhiễu lớn từ hệ điều hành.

• Điều đã hiểu rõ hơn sau khi làm: Hiểu rõ vai trò của mặt nạ bit trong khóa trạng thái (r, c, mask), cơ chế bỏ bản ghi cũ trong heap, lý do kiểm đích khi pop và cách chứng minh tính nhất quán khi ăn viên thức ăn cuối cùng.

Kiểm tra trước nộp	Tệp / người / kết quả thật
Báo cáo, tính tay, 6–8 slide	Đã hoàn thiện BaoCao.pdf, TinhTay.pdf (B1–B4) và ThuyetTrinh.pptx / Người kiểm: Nguyễn Quốc Hiếu
Mã Python, dữ liệu và README	Đầy đủ các tệp .py, du_lieu_nhom.json (seed 1526), README_NOP.md, DongGop.md / Người kiểm: Phan Ngọc Huy
24+12 cấu hình và hai trace H	Đã xuất bat_buoc.json (24), tu_tao.json (12), tay_H_UCS.json, tay_H_Avia.json / Người kiểm: Lê Đức Huy
10 test riêng, log thật	Đã chạy chay_kiem_thu.py với 10 test trong test_nhom.py, xuất ket_qua_test.txt (10/10 OK) / Người kiểm: Hoàng Trung Kiên
Giải nén chạy trên máy khác	Đã giải nén ZIP sang thư mục mới và chạy lại toàn bộ lệnh theo README_NOP.md / Người kiểm: Dương Nhật Huy
Năm người tập bảo vệ 15–20 phút	Cả 5 thành viên đã tập trình bày 5 phút, demo 3 phút và hỏi đáp 10 phút / Cả nhóm xác nhận

• Tên gói ZIP & Người kiểm gói sạch: Nhom15_AnHetThucAn_GiuaKy.zip .Người kiểm: Nguyễn Quốc Hiếu. Ngày kiểm: 29/10/2026.

• Năm thành viên xác nhận: Nguyễn Quốc Hiếu, Lê Đức Huy, Dương Nhật Huy, Phan Ngọc Huy, Hoàng Trung Kiên (Cả 5 thành viên xác nhận số liệu PDF khớp 100% với JSON/CSV/log).

Có thể thêm dòng, giãn ô. Giữ 6–8 trang nội dung chính, vết dài đưa vào phụ lục. Không xóa phần giải thích để chỉ giữ đáp số.